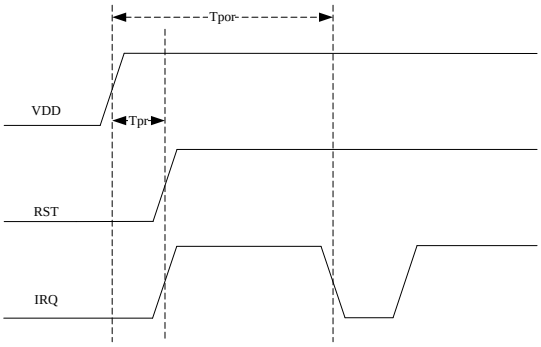
1. Power-up and reset timings

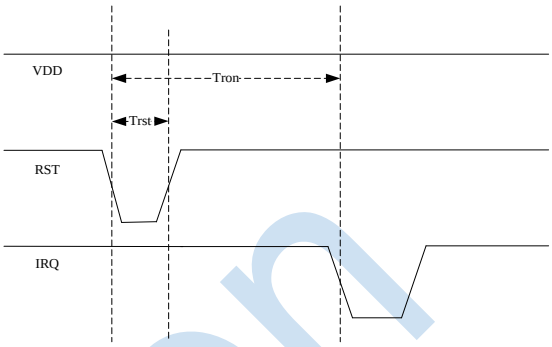
The chip has a built-in power-on reset circuit, so there is no need to connect a dedicated reset circuit externally.

The built-in power-on reset module will keep the chip in a reset state until the voltage is normal. The chip will also reset when the voltage falls below 2.6v.

When the external reset pin RST is low, the whole chip will be reset and the pin can be left floating.



Power-on reset sequence

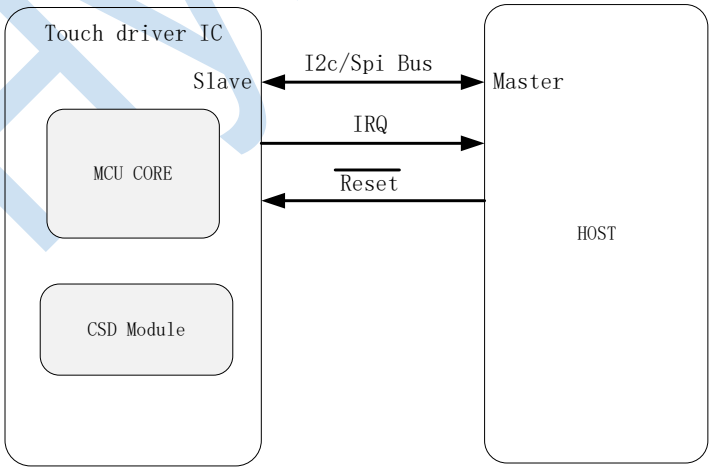


External Reset sequence

Parameter	Description	minimum	maximum	unit
Tpor	Chip initialization time after power-on	100	-	mS
Tpr	RST pin delayed pull-up time	5	-	mS
Tron	Chip reinitialization time after reset	100	-	mS
Trst	reset pulse time	0.1	-	mS

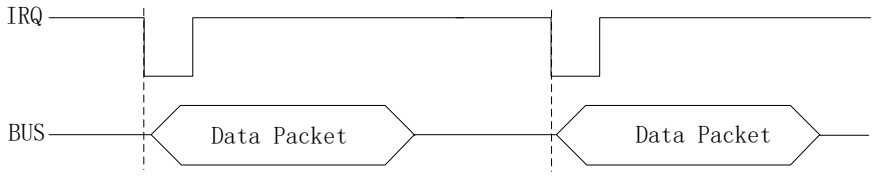
2. I2C/SPI interface description

communication interface



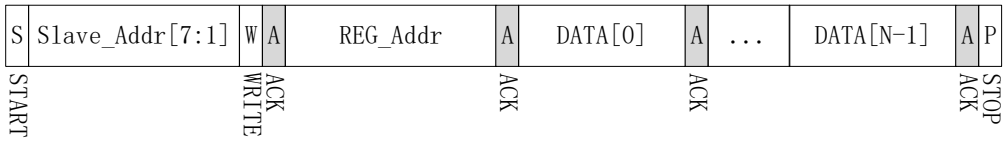
1) I2c/SPI communication for data transmission
2) The touch IC notifies via the IRQ pin and the host reads the touch information that needs to be reported The host resets the touch IC through the Reset pin
3) The I2c default slave device address is 0x58 (7bit). slave address can be configured by software, Please check with an engineer for specific projects
4) The SPI polarity default used Mode0， Maximum clock frequency 8MHz

IRQ falling edge-triggered communication

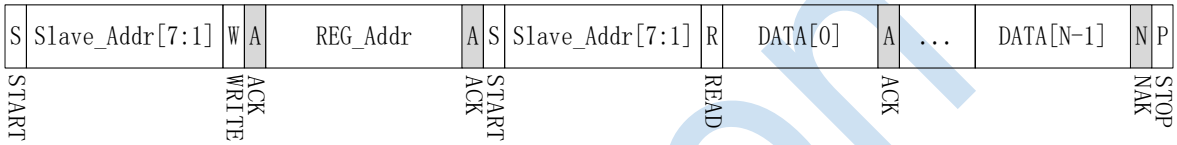


I2C read and write

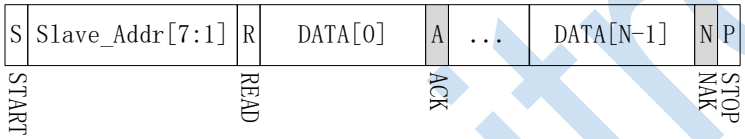
Write N bytes to the register address



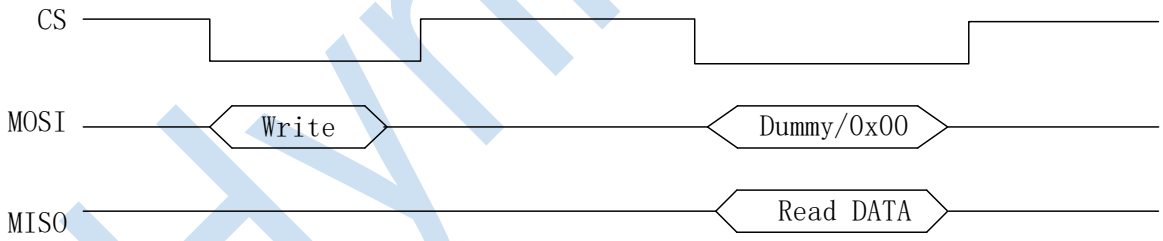
Read N bytes from register address



Append read N bytes



SPI read and write



3. Register Description

The IC works in nomal mode by default after powering up, if you don't need to switch the mode, you don't need to configure it.

Working mode switching command

Operating mode	commands	Description
nomal mode	Cmd[0]:0xD0 00 00 00 Cmd[1]:0xD0 00 01 00 Cmd[2]:0xD0 00 02 AB	Report Coordinates and key information
gesture mode	Cmd[0]:0xD0 00 0C 01	Report gesture ID
sleep mode	Cmd[0]: 0xD0 00 22 AB	Supend enter deepsleep

Nomal work mode register description

Reg_addr = Head + offset_L + offset_H eg:

head = 0xD007 offset = 0x00A5 , Reg_addr = 0xD007A500

head = 0xD007 offset = 0xA5 , Reg_addr = 0xD007A5

Head	offset	7	6	5	4	3	2	1	0	Description	Access
0xD007	0x0000	Checksum[7:0]								Start offset 0x0004 Length: num*5	R
	0x0001	Checksum[15:8]									R
	0x0002	Report type[7:0]								0xFF: pos 0xF0:gesture 0xE0:proximity	R
	0x0003	reserver				Report Finger_num					R
	0x0004	x_pos[7:0]									R
	0x0005	y_pos[7:0]									R
	0x0006	Z_press[7:0]									R
	0x0007	y_pos[11:8]				x_pos[11:8]					R
	0x0008	Finger event(2)				Finger id(1)				Nomal mode:Finger[0] Gesture mode:gesture id	R
		Gesture_id(3)									
	0x0009	x_pos[7:0]									R
	0x000A	y_pos[7:0]									R
	0x000B	Z_press[7:0]									R
	0x000C	y_pos[11:8]				x_pos[11:8]					R
	0x000D	Finger event				Finger id				Finger[1]	R
	……	……								……	R
	Pos_data	x_pos[7:0]									R
	Pos_data	y_pos[7:0]									R
	Pos_data	Z_press[7:0]									R
	Pos_data	y_pos[11:8]				x_pos[11:8]					R
	Pos_data	Finger event				Finger id				Finger[Finger_num-1]	R
0xD000	0x00	Data[0]								0x00:nomal mode 0xAB:factory mode	W
	0x01	Data[0]								0x00:nomal report 0xAB:dbg report	W
	0x02	Data[0]								0xAB:End of report	W
	0x03	Data[0]								0xAB:soft reset	W
	0x04	Data[0]								0x00: disable LP Scan 0xAB:enable LP Scan	W
	0x05	Data[0]								0x00: exit LP debug 0xAB:enter LP debug	W
	0x07	Data[0]								0x00:enable cycle sleep 0xAB: disable cycle sleep N:disable N cycle sleep	W
	0x0C	Data[0]								0x00:nomal report 0x01:gesture report	W
	0x22	Data[0]								0xAB:enter sleep mode	W

Explanation of superscript：

(1) Finger ID Starting from 0

(2) 1: Press 2: Hold 0: Lift

(3) 0:click 1: double-click 2: Scratch up 3: scratch down 4: scratch left 5: scratch right 6:'C' 7:'e' 8:'v' 9:'^' 10:'>' 11:'<' 12:'m' 13:'w' 14:'o' 15:'s' 16:'z' others:null

Read the point reference code

```
static int cst66xx_report(void)
{
    u8 buf[80],i=0;
    u8 finger_num = 0, len = 0,report_typ= 0;
    int ret = 0,retry = 2;
    while(retry--){ //Communication reading point data
        ret = hyn_wr_reg(hyn_66xxdata,0xD0070000,4,buf,9);  //switch to read point
        report_typ = buf[2];//FF:pos F0:ges E0:prox
        finger_num = buf[3]&0x0f;
        if(finger_num > 1 && finger_num < MAX_REPORT_NUM){
            len = (finger_num -1)*5;
            ret |= hyn_read_data(hyn_66xxdata,&buf[9],len);
        }
        if(hyn_sum16(0x55,&buf[4], finger_num *5) != (buf[0] | buf[1]<<8)){ //checksum
            ret = -1;
        }
        if(ret && retry) continue;
        ret = hyn_wr_reg(hyn_66xxdata,0xD00002AB,4,buf,0); //end of read
        if(ret == 0){
            break;
        }
    }
    //Start parsing the data.
    if(report_typ==0xff && finger_num >0){ //pos
        u16 index = 0,num = 0;
        for(i = 0; i < finger_num; i++)
        {
            index = i*5;
            hyn_66xxdata->rp_buf.pos_info[i].pos_id = buf[index+8]&0x0F;
            hyn_66xxdata->rp_buf.pos_info[i].event = buf[index+8]>>4;
            hyn_66xxdata->rp_buf.pos_info[i].pos_x = buf[index+4] + ((u16)(buf[index+7]&0x0F) <<8);
            hyn_66xxdata->rp_buf.pos_info[i].pos_y = buf[index+5] + ((u16)(buf[index+7]&0xf0) <<4);
            hyn_66xxdata->rp_buf.pos_info[i].pres_z = buf[index+6];
            if(hyn_66xxdata->rp_buf.pos_info[i].event) num++;
        }
        hyn_66xxdata->rp_buf.rep_num = num;
    }
    else if(report_typ == 0xF0){ //geture
        hyn_66xxdata->gesture_id = buf[8]&0xff;
    }
    return ret;
}
u16 hyn_sum16(int val, u8* buf,u16 len)
{
    u16 sum = val;
    while(len--) sum += *buf++;
    return sum;
}
```

4. Revision History

Version	Modification
V1.00	Initial release
V1.01	Fixed a problem with finger id descriptions